Next-Generation Raycasting: Integrating Advanced Verticality and AI-Driven Multi-Directional Sprite Generation

Introduction to Modern Pseudo-3D Engine Architecture

The intersection of classic 2.5D rendering techniques and modern generative artificial intelligence represents a paradigm shift in independent game development. Raycasting, the foundational rendering algorithm behind historic titles such as *Wolfenstein 3D* and *Doom*, relies on casting a series of rays from a simulated camera view into a two-dimensional grid or sector map to determine the distance to the nearest wall. By rendering vertical strips of texture scaled inversely to this distance, the algorithm simulates a three-dimensional perspective from purely 2D data[1].

While traditional raycasting engines are often mistakenly associated with strict, flat-plane limitations where characters are permanently bound to the ground, the algorithm can be mathematically extended to support variable sector heights, vertical camera movement (pitch), and true z-axis entity physics[4]. This permits the synthesis of a true 3D experience—complete with jumping, falling, and looking vertically—while maintaining the immense performance benefits and stylized aesthetic of a 2.5D engine[2].

Simultaneously, the demand for high-fidelity, multi-directional 2D sprites required to populate these vertically complex pseudo-3D environments has historically posed a massive bottleneck in asset production. To maintain the illusion of three-dimensional volume, an entity in a raycasted engine must be rendered from multiple angles—typically 8 or 16 distinct horizontal directional views, compounded by vertical pitch views when viewed from above or below[8].

The integration of the Model Context Protocol (MCP) with advanced Large Language Models (LLMs) such as Claude, alongside node-based generative AI workflows like ComfyUI, fundamentally resolves this bottleneck. By establishing automated pipelines capable of generating spatially and temporally consistent spritesheets directly within the integrated development environment, developers can execute complex, multi-directional pseudo-3D game architectures with unprecedented efficiency[11]. This analysis explores the exhaustive mathematical frameworks required to build a raycasting engine with full vertical mobility and sprite directionality, followed by a comprehensive architectural blueprint for leveraging generative AI systems to dynamically construct the necessary visual assets.

Mathematical Foundations of the Raycasting Engine

At its core, a raycasting engine operates on a simplified Cartesian coordinate system where the environment is represented as a two-dimensional map. The mathematical elegance of raycasting lies in its reduction of a three-dimensional rendering problem into a series of one-dimensional column calculations. This scales the computational rendering time by the

horizontal resolution of the viewport rather than the geometric complexity of the scene[1].

The Digital Differential Analyzer (DDA) Algorithm

The standard approach to grid traversal in a raycasting engine is the Digital Differential Analyzer (DDA) algorithm, known for its high efficiency on modern and legacy CPUs alike. The simulated camera is defined by a position vector (P_x, P_y) and a direction vector (D_x, D_y). To prevent visual distortion, specifically the spherical fish-eye effect caused by purely angular ray casting, a camera plane vector (C_x, C_y) is introduced, representing the projection surface orthogonal to the direction vector[14].

For each vertical column of pixels on the screen (denoted as x, ranging from 0 to the screen width), the engine calculates the initial ray direction by scaling the camera plane vector and adding it to the player's direction vector:

$$RayDir_x = D_x + C_x \cdot \left(\frac{2x}{Width} - 1\right)$$

$$RayDir_y = D_y + C_y \cdot \left(\frac{2x}{Width} - 1\right)$$

The algorithm then calculates the step directions (either 1 or -1) and the initial side distances from the player to the nearest horizontal and vertical grid boundaries. Variables typically denoted as $deltaDistX$ and $deltaDistY$ represent the distance the ray must travel from one x-side or y-side of a grid cell to the next[14].

The core DDA loop incrementally advances the ray's position across the grid by stepping either horizontally or vertically depending on which boundary is closest. When a solid grid cell boundary is intersected, the engine calculates the perpendicular wall distance to maintain perspective-correct scaling. This distance dictates the height of the vertical line drawn on the screen:

$$WallHeight = \frac{ScreenHeight}{PerpendicularDistance}$$

By drawing a vertical stripe of the appropriate texture, scaled by this height, the engine creates the illusion of depth[14].

Unprojection and Transformation Matrices

Scaling the engine to handle more complex geometry, variable viewing angles, and object billboarding requires unprojection and transformation matrices. Transforms are typically concatenated via 4x4 matrix arithmetic, where a spatial point is represented by a 4D vector $[X, Y, Z, 1]$ and a direction vector by $[D_x, D_y, D_z, 0]$ [15]. To form a ray from a specific screen position natively in 3D space, an unprojection process is performed on two positions with identical x and y coordinates but varying z-axis coordinates, mapping the 2D screen coordinate back into the 3D world space[16].

The upper 3x3 region of the transformation matrix dictates the world-space rotation of any given object or camera. Assuming $-z$ represents the forward direction in standard Cartesian OpenGL coordinates, negating the third row or column of the upper 3x3 matrix isolates the

forward directional vector. The resulting matrix values $[x_z, y_z, z_z]$ yield the absolute facing direction of an object, which is mathematically critical for determining how AI-generated sprites should be billboarded and which rotational frame must be selected relative to the camera[16].

Matrix Component	Dimension	Mathematical Function in Raycasting
Upper 3x3	3×3	Rotation; determines entity facing direction for sprite selection.
Translation Vector	1×3	Position; maps X, Y, Z world coordinates.
Perspective Divisor	W -axis	Enables scaling based on depth to simulate 3D projection.

Breaking the 2D Plane: Achieving True Verticality

The classical limitation of early raycasters, such as *Wolfenstein 3D*, was the strict adherence to a uniform floor and ceiling grid. Because the mathematics assumed a constant height for all walls and floors, the player character was permanently bound to the ground plane. Achieving true verticality—allowing characters to jump, fly, climb stairs, and view the environment from varying pitches—requires decoupling the rendering logic from a strictly uniform grid and introducing Z-axis mathematics, sector-based heights, and vertical shearing[2].

Sector-Based Rendering and BSP Trees

To allow characters to move freely in three dimensions, the environment must transition from a simple uniform grid to sector-based geometry. Pioneered by the *Doom* engine, this architecture defines the 2D map through vertices (2D points), which form linedefs (lines), which in turn enclose sectors (polygonal spaces)[3].

Crucially, each sector possesses an independent floor height and ceiling height parameter. This allows adjacent areas to represent steps, ledges, deep pits, or towering structures[2]. Because multiple floor heights exist, a true physical Z -axis is introduced into the engine's character controller logic. A character is assigned a specific Z -coordinate and a Z -velocity vector. Gravity is applied to the velocity each frame, and collision detection clamps the character's Z -coordinate to the floor height of the specific sector they currently occupy[5]. This fundamentally fulfills the requirement for characters to move dynamically through 3D space rather than

remaining artificially grounded.
To render these complex variable-height sectors efficiently without a hardware depth buffer, classic engines rely on Binary Space Partitioning (BSP) trees. The BSP compiler recursively divides the map geometry using splitting planes until all remaining geometry is contained within convex subsectors[5].
During execution, the rendering pipeline traverses the BSP tree starting from the root node. By evaluating a dot product to determine which side of the node's dividing line the camera currently occupies, the algorithm draws the child nodes recursively. This guarantees that geometry is processed in strict front-to-back or back-to-front order[3]. As walls are drawn, their varying top and bottom Y -screen coordinates define bounding areas known as visplanes, which are subsequently filled with floor and ceiling textures[18].

Y-Shearing and Pitch Manipulation

While sector heights provide physical spatial verticality, giving the player the ability to look up and down (altering camera pitch) requires visual manipulation. Because pseudo-3D raycasting inherently relies on drawing vertical columns orthogonal to the floor, physically rotating the camera's directional vector along the pitch axis mathematically invalidates the 1D column rendering, resulting in catastrophic perspective distortion[2].
To circumvent this, 2.5D engines employ a software rendering technique known as Y-shearing, or horizon shifting[20]. Rather than altering the underlying ray geometry, Y-shearing shifts the vertical drawing coordinates of the rendered columns up or down the 2D screen buffer based on the player's simulated pitch angle.
Let H represent the base horizon line (typically $ScreenHeight/2$). The rendering offset is calculated as:

$$HorizonOffset = \tan(Pitch) \cdot FocalLength$$

This calculated offset is added to the starting and ending y -coordinates of every vertical wall stripe and every sprite being rendered to the viewport[20]. If the player looks up, the horizon line is shifted downward; if the player looks down, the horizon is shifted upward. While severe pitch angles (e.g., looking straight up at 90 degrees) will reveal the artificial stretching of the vertical textures, moderate angles perfectly synthesize the ability to look around a 3D vertical space without requiring polygon rasterization[21].

Sprite Occlusion in Pseudo-3D Space

Integrating 2D sprites into a vertically complex environment necessitates advanced occlusion techniques. In a flat engine with uniform wall heights, a simple 1D array recording the perpendicular distance of the wall drawn at each vertical column is sufficient to clip sprites[23]. If the sprite's distance is greater than the array's stored value at column X , that vertical slice of the sprite is omitted.
However, when variable wall heights and Y-shearing are introduced, a sprite might be partially occluded by a low wall (like a railing or a step) but remain fully visible above it. A standard 1D

Z-buffer cannot resolve this vertical complexity because occlusion now depends on both X and Y coordinates simultaneously[24].

To solve sprite visibility with variable heights, modern raycasting engines adopt advanced clipping architectures:

Occlusion Architecture	Mechanism	Computational Complexity	Use Case
Drawsegs / Visplanes	Dynamically modifies sprite screen-space bounding boxes against stored visible solid wall segments.	Medium (Software)	Classic sector engines (*Doom* style), software rendering[3].
Hardware Depth Maps	A true 2D depth buffer stores the depth of every pixel. Sprite pixels are depth-tested against the buffer before rendering.	High (Software) / Low (Hardware)	Modern hardware-accelerated 2.5D engines (GZDoom, Unity)[24].
Frame Buffer Objects (FBOs)	Geometry is drawn to a depth buffer; sprites are rendered into a separate texture, blended via depth testing to discard obscured pixels.	Low (Hardware)	Advanced OpenGL ports requiring complex transparent overlapping[25].

By implementing a 2D depth buffer or utilizing Drawseg mathematical clipping, the engine allows AI-generated sprites to exist convincingly within the varied geometry. A sprite character can stand behind a waist-high crate, move up a staircase, and accurately clip against the varied sector heights.

Trigonometry of Multi-Directional Sprites

For a 2D entity to convincingly replicate 3D volume within this vertically varied space, the entity must possess high-fidelity directional awareness. A single static image of a character facing

forward breaks the spatial illusion as the player navigates around them. The engine must utilize billboarding alongside angle-dependent texture swapping to ensure the sprite appropriately reflects its spatial relationship to the camera[23].

Billboarding Mechanics

Billboarding is the mathematical process by which a 2D plane (the sprite) is dynamically rotated to always face the camera's orthogonal plane, preventing the image from appearing paper-thin when viewed from an angle[26]. In a raycasting environment, this is achieved by transforming the sprite's world coordinates relative to the player's camera matrix.

To render the sprite, the engine calculates a displacement vector from the player to the sprite:

$$Sprite_x = WorldPos_x - Player_x$$

$$Sprite_y = WorldPos_y - Player_y$$

These coordinates are multiplied by the inverse of the camera's 2x2 transformation matrix to translate the sprite into the camera's localized view space. The resulting localized X and Y values determine the sprite's horizontal placement on the screen and its scaled height and width relative to its distance[22].

Angle Calculations for 8-Way and 16-Way Directionality

To render an entity that appears to exist in three dimensions, the sprite asset must contain multiple frames representing different horizontal viewing angles—commonly 8-way (45-degree increments) or 16-way (22.5-degree increments) directionality[8].

Determining which frame to display relies on calculating the relative angle between the entity's forward-facing world vector and the vector drawn from the entity to the player's camera. The fundamental mathematics for this calculation utilizes the two-argument arctangent function (atan2), which perfectly resolves the angle across all four Cartesian quadrants[17]:

$$\theta_{player} = \text{atan2}(PlayerPos_y - EntityPos_y, PlayerPos_x - EntityPos_x)$$

$$\theta_{entity} = \text{EntityYaw}$$

The relative viewing angle is the difference between the angle to the player and the entity's current physical rotation in the world geometry:

$$\Delta\theta = \theta_{player} - \theta_{entity}$$

To normalize this continuous angle into an 8-way directional system, the result is shifted, wrapped between 0 and 360 degrees (or 0 to 2π radians), and divided into distinct, quantized index buckets:

$$Index = \text{floor}\left(\frac{(\Delta\theta + 22.5^\circ) \bmod 360^\circ}{45^\circ} \right)$$

If the index evaluates to 0, the player is looking at the entity's front. If the index is 4, the player is viewing the entity from directly behind. This integer index is subsequently used to offset the texture lookup into the generative AI-produced spritesheet[26].

Elevation and Pitch States for Sprites

Because the engine supports full Z -axis mobility, horizontal yaw calculations are insufficient. Viewing a character from a high elevation (e.g., aiming down from a balcony) should theoretically reveal the top of their head and shoulders, whereas viewing from below should reveal the underside of their geometry.

To synthesize true 3D spatial awareness using 2D sprites, pitch states must be integrated[10]. The elevation angle is calculated using the relative Z -axis delta, solving for pitch:

$$\phi = \text{atan2}\left(PlayerPos_z - EntityPos_z, \sqrt{(PlayerPos_x - EntityPos_x)^2 + (PlayerPos_y - EntityPos_y)^2}\right)$$

By evaluating the elevation angle ϕ, the engine determines if the camera is level, elevated, or depressed relative to the entity. An advanced pseudo-3D engine cross-references the 8-way yaw index with a 3-way pitch state (e.g., Level, High-Angle, Low-Angle).

This requirement triggers an exponential geometric explosion in required assets. A standard character requires 8 horizontal directional frames. Adding upper and lower pitch states triples this to 24 frames for a single static pose[10]. Factoring in standard animation states—Idle (4 frames), Walk (8 frames), Attack (6 frames)—a single entity now requires over 400 distinct, spatially perfectly consistent frames of animation. Historically, this volumetric asset requirement made multi-pitch directional sprites economically unfeasible for independent developers. However, modern generative AI pipelines have completely inverted this paradigm, enabling the rapid and automated generation of massive directional spritesheets[13].

Generative AI Asset Pipelines via Model Context Protocol (MCP)

To populate this highly specific rendering environment, developers must utilize generative AI tools. Specifically, the Model Context Protocol (MCP) allows LLMs like Claude to generate the multi-directional sprites and spritesheets autonomously[12].

MCP is an open-standard communication layer that allows AI assistants to interface dynamically with external systems, REST APIs, and local file structures without requiring developers to write custom, hardcoded toolsets for every interaction[11].

Integrating AI Agents with Engine Development

Through MCP, an LLM functioning in a coding environment (such as the Claude Code CLI or an integrated IDE like Cursor) can write the C++ or Python code defining the raycasting engine while proactively executing API calls to generate the visual assets required by the codebase[12].

To configure this, a developer adds an external MCP server to their workspace. For example, connecting Gamelabs Studio or AutoSprite involves a command-line execution such as:

claude mcp add --transport sse gamelab-mcp http://api.gamelabstudio.co:8765/sse --header "X-API-Key: YOUR_KEY" --scope local[12].

Using --scope local ensures the connection is restricted to the specific project configuration, securely passing API tokens without committing them to version control[12]. Once established over Server-Sent Events (SSE) or HTTP, the developer issues unified, natural language prompts:

"Create an 8-direction walking animation for a heavy cyber-monk enemy, remove the

background, compile it into a spritesheet, and write the engine logic to load the resulting JSON atlas."[12].

Through the MCP specification, Claude parses the natural language, identifies the necessary generation parameters, and interfaces with the specialized asset generators to orchestrate the creation of hundreds of discrete frames[12].

Dedicated AI Spritesheet Generators

Platforms like Gamelabs Studio and AutoSprite function as dedicated AI-driven asset pipelines optimized specifically for 2D game engines[33]. These systems differ significantly from general-purpose image generators (like Midjourney) by prioritizing strict spatial coherence, looping animation sequencing, and alpha-channel preservation.

When processing a complex multi-directional request via MCP, these specialized engines perform a highly specific sequential operation:

1. **Base Generation:** A master rendering of the entity is created from a specified angle based on the prompt using a fine-tuned image model (e.g., Nano Banana Pro)[35].
2. **Multi-View Synthesis:** The AI utilizes specialized ControlNets and multi-angle projection models to extrude the base generation into multiple cohesive viewing angles simultaneously[35].
3. **Animation Extrapolation:** For active animation states, advanced video-to-image AI tools (such as LTX-2.3, Wan2.2, or Seedance) generate temporally stable motion bursts. These sequences are analyzed, stabilized, and sliced into discrete looping frames[36].
4. **Chroma-Keying and Alpha Channeling:** The AI isolates the subject, utilizing edge-detection to strip the background. Crucially for visual effects in a raycaster, this preserves the semi-transparent pixels necessary for anti-aliasing and particle effects, maintaining true translucency without hard artifact edges[34].
5. **Atlas Generation:** The frames are algorithmically packed into a uniform atlas grid (e.g., 256x256 per frame). A supplementary JSON metadata file is generated containing frame bounds, grid layout, and timing[33].

The MCP layer automatically retrieves the resulting .png and .json files and injects them directly into the developer's local project directory, effectively eliminating the context-switching normally associated with technical art creation[39].

Advanced Workflows: 3D to 2D AI Rendering pipelines

The primary challenge in AI asset generation for games remains style and spatial consistency. As observed in early AI integration attempts, generating a character facing forward and then generating the "same" character facing left often resulted in disparate armor designs, shifting color palettes, and morphological inconsistencies—phenomena that shatter the illusion of a coherent 3D entity when rotating in a raycaster[13].

For developers requiring absolute frame-perfect consistency—particularly when integrating the aforementioned vertical pitch frames—pure text-to-image AI often struggles to maintain geometric stability across the 24 to 40 angles required. The optimal architecture synthesizes a rudimentary 3D workflow with sophisticated AI image processing via ComfyUI.

Automated Blender Rendering Scripts

To ensure flawless geometric consistency across all 8 or 16 directional views, developers can construct a low-fidelity "blockout" 3D model of the character[40]. This model does not require complex textures or detailed modeling; it serves purely as a rigid spatial proxy.

A Python script executed within a 3D suite like Blender automates the rendering process. The script programmatically repositions a digital camera around the 3D proxy at precise 45-degree rotational increments along the yaw axis[41]. Because the engine requires vertical pitch states, the script elevates the camera by 45 degrees, repeats the 360-degree rotation, and then depresses the camera to capture the low-angle views[44].

The output is a sequence of highly accurate, perfectly aligned silhouette or depth-map frames covering every spatial direction and pitch relative to the entity[46].

ComfyUI and ControlNet Implementations

These rudimentary 3D renders are ingested into ComfyUI, a node-based graphical interface for advanced Stable Diffusion pipelines[35]. ComfyUI allows the developer to construct specific generation chains that strictly guide the AI's rendering process.

The fundamental node in this workflow is ControlNet, a neural network architecture designed to control diffusion models by adding extra spatial conditions. The automated 3D renders from Blender are passed through two distinct ControlNet preprocessors:

1. **Depth Maps (Z-Depth):** Translates the 3D volume of the proxy into grayscale values, dictating to the AI exactly how far forward or backward specific limbs and features reside in space[48].
2. **Normal Maps:** Dictates the surface curvature of the proxy geometry, allowing the AI to calculate physically accurate lighting, shadowing, and material properties across the sprite[48].

By constraining the AI generation with these ControlNet layers, the AI acts as a sophisticated, real-time texture artist rather than a blind generator. It applies complex stylistic elements (e.g., highly detailed pixel art, high-fantasy oil painting, or sci-fi aesthetics) directly over the perfectly consistent 3D geometry[49].

Because the underlying spatial data is derived from a mathematically rigorous 3D render, the resulting AI-generated sprites seamlessly align when the raycasting engine pivots between the 8 directional views in the pseudo-3D environment[45].

Synthesizing the Architecture: Game Engine Implementation

To execute the user's vision—a fully vertical raycasted engine populated with AI-generated, multi-directional sprites—the architectural pipeline must synchronize the automated asset generation with the mathematical rendering logic.

1. Data Structure and Sprite Atlas Management

When an MCP-driven LLM or ComfyUI pipeline generates an entity, it provides a unified sprite atlas containing all directional and pitch frames. The engine must parse the accompanying JSON file to map logical animation states to specific UV coordinates on the sprite texture[33].

A standard mapping schema for an advanced raycaster involves a multi-dimensional array lookup, structured to account for the angular mathematics discussed previously:
TextureID = SpriteAtlas[AnimationState][PitchIndex][YawIndex][AnimationFrame]
Where parameters evaluate as:

- AnimationState: Idle, Walking, Attacking.
- PitchIndex: 0 (Viewing from below), 1 (Level), 2 (Viewing from above).
- YawIndex: 0 through 7 (The 8 quantized cardinal and ordinal directions).
- AnimationFrame: The current temporal frame of the looping motion.

2. The Rendering Pipeline Sequence

The final rendering loop executed per-frame synthesizes all physical and visual logic:

1. **Physics and Environment Traversal:** The engine processes the player's 3D coordinates (X, Y, Z) and 3D look-vector. The DDA algorithm casts rays through the 2D grid to locate boundaries[14]. Gravity is applied to the Z-velocity, updating spatial positioning[5].
2. **Sector Height Calculation:** The rays interact with BSP sector definitions to determine varied floor and ceiling heights. Y-shearing screen offsets are calculated based on the simulated camera pitch[5].
3. **Depth Buffer Population:** The visible wall segments are rendered to the viewport, and a 2D Z-buffer is concurrently updated with the depth values of the geometry to ensure pixel-perfect future occlusion[24].
4. **Entity Iteration:** The engine iterates through all active entities present in the current visible subsectors[3].
5. **Spatial Calculation:** For each entity, the true 3D Euclidean distance is computed. The entity's world coordinates are multiplied by the inverse camera matrix to determine their screen X position and scaled visual dimensions[22].
6. **Directional Math:** The relative horizontal yaw angle ($\Delta\theta$) and vertical elevation angle (ϕ) are calculated using atan2 functions relative to the player's current viewpoint[17].
7. **Asset Retrieval:** The angles are quantized into the 8-way index and 3-way pitch index. The engine retrieves the correct AI-generated frame from the loaded JSON texture atlas[29].
8. **Depth Tested Drawing:** The chosen sprite frame is drawn column by column, vertically scaled to match the visual perspective. Each pixel is cross-referenced with the 2D Z-buffer; if the sprite pixel is further away than the stored sector geometry (e.g., hidden behind a chest-high wall or the lip of a higher sector floor), it is clipped and discarded[24].

3. Scaling Game Production with AI

By establishing this comprehensive technical pipeline, the heavy lifting associated with traditional independent engine development is negated. When a new enemy archetype is required, the developer utilizes the MCP interface in their codebase to trigger the AI workflow. The AI generates the required 400+ frames of animation across all spatial dimensions[33], updates the JSON mapping file, and autonomously instances the entity class in the engine

code.
The raycasting engine, built on robust sector-based geometric rules and precise angular trigonometry, receives these assets and effortlessly maps them into a dynamic, pseudo-3D world where characters move, jump, and traverse complex vertical layouts.

Conclusion

The creation of a 2D raycasting game that achieves the spatial volume and vertical freedom of a true 3D engine is a highly feasible architectural goal. By integrating advanced techniques such as Y-shearing for pitch simulation, true Z -axis physics bounded by sector-based heights, Binary Space Partitioning for rapid traversal, and 2D depth buffering for pixel-perfect sprite occlusion, developers can push the algorithmic boundaries far beyond the flat planes of early 1990s shooters.
Crucially, the historic barrier to entry for this volumetric visual style—the exponential requirement for multi-directional and multi-pitch sprite frames—has been fundamentally dismantled by modern generative AI. Through the integration of the Model Context Protocol (MCP), Large Language Models can dynamically invoke specialized asset pipelines like Gamelabs Studio and AutoSprite directly within the development environment, managing complex metadata output seamlessly. When absolute geometrical perfection is required, automated 3D-to-2D pipelines utilizing Python Blender scripting and ComfyUI ControlNet ensure that every 8-way rotational sprite remains spatially locked and cohesive.
The synthesis of highly optimized pseudo-3D mathematics with these next-generation AI asset pipelines yields a robust, scalable engine architecture. This paradigm allows solo developers and small teams to execute complex, atmospheric game worlds with unprecedented speed, fidelity, and technical depth.

Works cited

1. Understanding Raycasting Step-by-Step - Andor Saga, https://andorsaga.wordpress.com/2014/01/22/understanding-raycasting-step-by-step/
2. Doom Ran on a 486 — Here's How They Made That Possible, https://harlepengren.com/how-doom-engine-worked/
3. The Doom rendering engine, https://soulsphere.org/apocrypha/doom-engine/
4. Introduction to the camera view - Unity 6.6 User Manual, https://docs.unity3d.com/6000.1/Documentation/Manual/UnderstandingFrustum.html
5. Doom rendering engine - Doom Wiki - Fandom, https://doom.fandom.com/wiki/Doom_rendering_engine
6. How the Doom engine works | Goom, https://secondchoice.github.io/Goom/
7. Ruby on Doom - GitHub, https://github.com/khasinski/doom
8. 8-Direction Sprite Generator – AI Character Turnaround - Sprite Flow, https://spriteflow.io/direction-sprite-generator
9. How to Make an 8-Direction Sprite Sheet with AI | Ludo.ai Docs, https://ludo.ai/docs/sprite-generator/8-direction-sprites

10. Native support for pitch rotations for sprites. - the ZDoom Forums!, https://forum.zdoom.org/viewtopic.php?t=78252
11. Introducing MCP · Building with the Claude API, https://academy.claude.com/courses/building-with-the-claude-api/introducing-mcp
12. How to setup Gamelabs Studio MCP with Claude Code, https://gamelabstudio.co/blog/how-to-generate-spritesheet-claude-code
13. Addressing AI Use in Game Development - Darko Unity, https://darkounity.com/blog/addressing-ai-use-in-game-development
14. Notes on ray casting - nielssp.dk, https://nielssp.dk/2024/11/notes-on-raycasting
15. Ray casting - Wikipedia, https://en.wikipedia.org/wiki/Ray_casting
16. Physics - Raycasting, https://research.ncl.ac.uk/game/mastersdegree/gametechnologies/physicstutorials/1raycasting/Physics%20-%20Raycasting.pdf
17. How To Calculate Pitch For A Projectile Affected By Gravity, https://forum.zdoom.org/viewtopic.php?t=55277
18. Doom Engine source code review - Fabien Sanglard, https://fabiensanglard.net/doomIphone/doomClassicRenderer.php
19. 17 Rendering: Overview – Introduction to the Doom Repository, https://bookdown.org/robertness/doom_tour/7_1_rendering_overview_193.html
20. Translating a y-shearing horizon offset to a pitch angle in a raycaster, https://stackoverflow.com/questions/69485992/translating-a-y-shearing-horizon-offset-to-a-pitch-angle-in-a-raycaster
21. flutter_scene changelog | Flutter package - Pub.dev, https://pub.dev/packages/flutter_scene/changelog
22. gzdoom/src/rendering/r_utility.h at master - GitHub, https://github.com/ZDoom/gzdoom/blob/master/src/rendering/r_utility.h
23. Raycasting III: Sprites - Lode Vandevenne, https://lodev.org/cgtutor/raycasting3.html
24. A solution to sprite visibility in a raycasting engine with variable wall, https://computergraphics.stackexchange.com/questions/10918/a-solution-to-sprite-visibility-in-a-raycasting-engine-with-variable-wall-height
25. Sprite Clipping in Strife: Veteran Edition Explained - Doom64 EX, https://doom64ex.wordpress.com/2014/12/24/sprite-clipping-in-strife-veteran-edition-explained/
26. Devlog #2 — Rendering sprites - by Jakub Neruda - Medium, https://medium.com/@nerudaj/devlog-2-rendering-sprites-81a378575f76
27. Corgi Engine Releases, https://corgi-engine.moremountains.com/corgi-engine-releases
28. Doom-like angle based sprite changing - Stack Overflow, https://stackoverflow.com/questions/22623013/doom-like-angle-based-sprite-changing
29. 8 Directional sprite in 3d world, how to retrieve angle?, https://discussions.unity.com/t/8-directional-sprite-in-3d-world-how-to-retrieve-angle/63611
30. Introducing the Model Context Protocol - Anthropic,

https://www.anthropic.com/news/model-context-protocol
31. GameLabs Studio MCP - AI Sprite & Game Asset Toolkit - LobeHub, https://lobehub.com/nl/mcp/macuseri686-gamelab-studio-mcp
32. Understanding Model Context Protocol (MCP) : A Full Deep Dive +, https://medium.com/ai-cloud-lab/model-context-protocol-mcp-with-ollama-a-full-deep-dive-working-code-part-1-81a3bb6d16b3
33. Claude Code Sprite Sheet Generator - AutoSprite, https://www.autosprite.io/claude
34. AI SPRITE SHEET GENERATOR - ANIMATE SPRITES IN SECONDS, https://gamelabstudio.co/ai-sprite-sheet-generator
35. Sprite Sheet Generator Workflow - Comfy, https://comfy.org/hub/workflows/sprite-sheet-generator-fe5600667e2c/
36. ComfyUI Workflows: Multi-Step AI Image & Video Workflows - Comfy, https://comfy.org/hub/workflows/
37. Sprite Sheet Generator - ComfyUI Workflow, https://comfy.org/workflows/sprite-sheet-generator-fe5600667e2c/
38. How to generate spritesheets for Construct engine with AI, https://gamelabstudio.co/blog/how-to-spritesheet-construct-ai
39. Generate spritesheets and animations for your game with Gamelab, https://www.reddit.com/r/aigamedev/comments/1pdd38n/generate_spritesheets_and_animations_for_your/
40. 3D Model to Sprite Sheet: 8 Direction Sprites from GLB/FBX - The3DAI, https://the3dai.com/3d-to-sprite-sheet
41. Blender Sprite Renderer - Nibbleblog, https://nibble.blog/posts/sprite-generator/
42. Render 4 or 8 Direction Animated Sprites from Blender Tutorial by, https://foozlecc.itch.io/render-4-or-8-direction-sprites-from-blender
43. Render 8 Direction Animated Sprites in Blender with Python, https://www.youtube.com/watch?v=l1lo7fLYV4o
44. How can I export a blender model as a sprite sheet?, https://blender.stackexchange.com/questions/5829/how-can-i-export-a-blender-model-as-a-sprite-sheet
45. Looking for help on automating workflow 2d sprites, 8 directions, https://www.reddit.com/r/learnblender/comments/si3ovs/looking_for_help_on_automating_workflow_2d/
46. An Automated Sprite Rendering System using Blender, https://mirror.math.princeton.edu/pub/gimp/gimp/lgm/2014/day_2/033_An_Automated_Sprite_Rendering_System_using_Blender/033_An_Automated_Sprite_Rendering_System_using_Blender.pdf
47. Directional blender python script (4,8,16,32) with a rust spritesheet, https://github.com/Maghwyn/blender_directional_spritesheets
48. I built a workflow to generate ultra-controllable AI images from 3D, https://motion-live.com/articles/I-built-a-workflow-to-generate-ultracontrollable-AI-images-from-3D-renders/
49. These FREE 3D AI Workflows are INCREDIBLE! (ComfyUI Tutorial +, https://www.youtube.com/watch?v=o5ZqrVNoeiI

50. Now AI Builds ComfyUI-Style Workflows for 3D Characters ... - Reddit, https://www.reddit.com/r/TopologyAI/comments/1uybpf0/now_ai_builds_comfyuistyle_workflows_for_3d/
51. What is a good Comfy AI workflow for creating animated sprites?, https://www.reddit.com/r/aigamedev/comments/1vruo3l/what_is_a_good_comfy_ai_workflow_for_creating/
52. 3d to Sprite Studio : r/gamedev - Reddit, https://www.reddit.com/r/gamedev/comments/1wkleyb/3d_to_sprite_studio/
53. Ray-Casting [C] - Medium, https://ismailassil.medium.com/ray-casting-c-8bfae2c2fc13
54. Doom-style sprite "rotation" doesn't distribute properly ... - Reddit, https://www.reddit.com/r/godot/comments/1mi5msz/doomstyle_sprite_rotation_doesnt_distribute/